Kick Start 2022 - Coding Practice with Kick Start Session #1

Analysis: Centauri Prime

View problem and solution walkthrough video

Given the name of the kingdom, we need to determine the ruler of that kingdom. Based on the information given in the problem, we can say that who the ruler is depends on the last letter of the kingdom name.

- If the last letter is a vowel, then the kingdom is ruled by Alice.
- If the last letter is a consonant other than 'y', then the kingdom is ruled by Bob.
- If the last letter is `y`, then the kingdom is ruled by `nobody`.

Test Set 1

Let us create a function `getRuler` which takes as input a string kingdom and returns whether it is ruled by `Alice`, `Bob` or `nobody`. Consider x to be the last letter of $\mathbf{kingdom}$.

- We will first check if x is a vowel, for this we can create a hashset consisting of all the vowels i.e. `{a, e, i, o, u}` and check if x is present in the hashset. If it is present return `Alice`.
- Now let us check if x is equal to `y` if so, then return `nobody`
- If both the above conditions are not satisfied, then x is a consonant and not equal to `y`. In this case, return `Bob`.

Test Set 2

We can use the same approach for this test as well, but there is a small catch. In Test Set 1, because of the constraints, $\mathbf{kingdom}$ has at least 3 letters so x is always lowercase. In Test Set 2, $\mathbf{kingdom}$ can have only one letter, so x is an uppercase. We need to convert it to lowercase and then check if it is a vowel or equal to `y`.

Time Complexity

Creating a hashset of vowels: $O(5)$, which is equivalent to $O(1)$.
For each test case:
Converting x to a lowercase in case of $\mathbf{kingdom}$ having only one letter: $O(1)$.
Checking if x is a vowel: $O(1)$.
Checking if x is equal to `y`: $O(1)$.
We have $\mathbf{T}$ test cases, hence the overall time complexity is $O(\mathbf{T})$

Note

Please take care of adding a `.>` (terminating period) at the end of the output sentence.

Kick Start 2022 - Coding Practice with Kick Start Session #1

Analysis: H-index

View problem and solution walkthrough video

Test set 1

For a set of papers, let us define a function $score(x)$, which denotes the number of papers in the set with citations at least x. We can calculate the score for every integer $x \leq n$, where n is the number of papers in the set. For each set of papers, the answer is the maximum x such that $score(x) \geq x$. We can store the counts of each of the citation numbers in the input in an array, and calculate cumulative sum to find the score for any x.

We need to find the maximum x where $score(x) \geq x$, each time a new paper is added. We can find that using the approach mentioned above in $O(max(\mathbf{A}))$ time, where $max(\mathbf{A})$ is the maximum citation number in the set of papers. An observation here is that we can consider all citations $\geq \mathbf{N}$ as $\mathbf{N}$, which allows us to find H-index for any set of papers in $O(\mathbf{N})$. This leads to a total time complexity of $O(\mathbf{N}^2)$ per test case, which is sufficient for test set 1.

Test set 2

For test set 2, we initialize the H-index with 0, and after adding each paper, we need to update the current H-index. We can use a minimum priority queue data structure to store the citation numbers. As we go through each of the papers, we keep updating the current H-index. We also keep updating the priority queue so that it only contains numbers greater than the current H-index and remove the rest. For each new paper, we can follow these steps:

1. If the current citation number is bigger than the current answer, add it in the priority queue
2. Remove all the citation numbers from the priority queue which are not greater than the current answer
3. If the size of priority queue is not less than the current answer + 1, increment the current answer by 1 and return to step 2

A single operation of priority queue takes $O(\log \mathbf{N})$ time, and each number is added and removed at most once. This leads to a total time complexity of $O(\mathbf{N} \log \mathbf{N})$ per test case, which is sufficient for test set 2.

A solution with linear time complexity is also possible, if we store the citation numbers in an array or a hashtable to calculate the answer. This is left as an exercise for the readers.

Kick Start 2022 - Coding Practice with Kick Start Session #1

Analysis: Hex

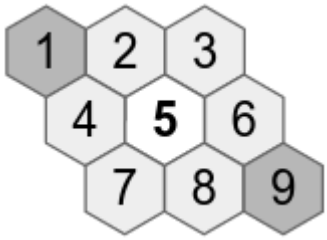
View problem and solution walkthrough video

Given an $\mathbf{N} \times \mathbf{N}$ grid representing a rhombus-shaped board with hexagonal cells, we have to output one of four possible board states: `Impossible`, `Red wins`, `Blue wins`, or `Nobody wins`.

The square input grid does not fully indicate what it means for cells to be connected. For example, consider the cells labeled `1` through `9` in this 3×3 grid:

```
123
456
789
```

Compare the equivalent cells in the game board:

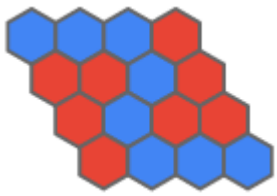


The central cell `5` is not connected to cell `1` or cell `9`. It is connected to cells `2`, `3`, `4`, `6`, `7`, and `8`. This adjacency rule applies throughout the solution.

Another important observation is that the game board is not symmetrical for the Red and Blue players. This is not obvious from the square grid input. For example, consider this 4×4 grid:

```
BBBR
RRBR
RBRR
RBBB
```

The text representation looks symmetrical for Red and Blue. However, the equivalent portion of the game board looks like this:



Notice that the blue stones in the center are connected, but the red stones are not.

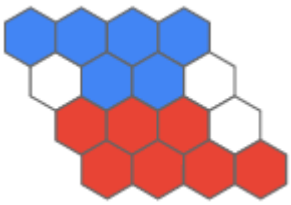
Test set 1

In test set 1, we are given cases with $1 \leq \mathbf{N} \leq 10$. We can use flood fill to find connected sets of cells. We start by considering the blue cells at the west side of the board. From a given blue cell, we visit each connected cell to see if it is also blue. We keep track of which cells have been visited so that we do not examine the same cell more than once.

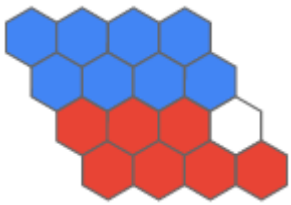
If the flood fill reaches a blue cell at the east side of the board, we know that there is a winning path for the Blue player. If this is not the case, we flood-fill red cells starting from the north side of the board and check whether we reach the south side. This tells us whether the Red player has a winning path. (It is not physically possible for both Blue and Red to have a winning connection, so we do not consider that case.)

Because we traverse the entire $\mathbf{N} \times \mathbf{N}$ board, the time complexity of the flood fill is $O(\mathbf{N}^2)$. However, the solution is not complete yet. We have to think about what would make a board state impossible.

The following state is impossible according to the game rules. By counting the stones, we can see that Red must have placed a stone after Blue won:



Another kind of impossible state is when a player has a winning path, but there is no single stone that could have been placed in the final move to change the board from a non-winning state to a winning state. Consider the following board:



Blue has a winning path. Now consider what state the board was in before Blue's final move. No matter which blue stone we remove from the board, Blue still has a winning path. This tells us that the game did not end according to the rules, so we say that the board is in an `Impossible` state.

Bringing everything together, we can solve test set 1 as follows.

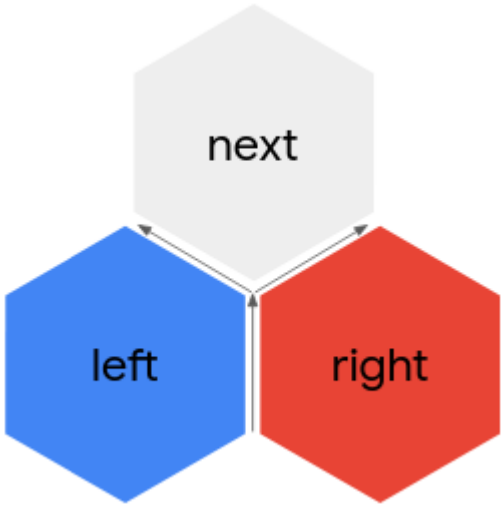
1. Count the stones placed by each player and make sure they took no more moves than allowed, or else the state is `Impossible`.
2. Use flood fill to look for a path of blue cells from the west side of the board to the east side.
3. If there is a blue path, check whether the board state is possible. For each blue stone on the board, temporarily remove the stone and use flood fill again. If there is still a blue path in the temporary state for every single blue stone, the board state is `Impossible`. Otherwise, the state is `Blue wins`.
4. Perform steps 2 and 3 for the Red player, looking for a path of red cells from the north side of the board to the south side.
5. If there is no blue path or red path, the state is `Nobody wins`.

In step 3, for each of $O(\mathbf{N}^2)$ stones, we run flood fill in $O(\mathbf{N}^2)$ time, making a total time complexity of $O(\mathbf{N}^4)$.

Test set 2

In test set 2, we are given cases with $1 \leq \mathbf{N} \leq 100$. An $O(\mathbf{N}^4)$ solution may be too slow for $\mathbf{N} = 100$.

A more efficient approach involves tracing paths *between* cells. Imagine that we are walking between two cells with a blue stone on our left and a red stone or empty cell on our right:



There is a cell ahead of us. If the next cell has a blue stone on it, we go right. Otherwise, we go left. We continue to traverse the grid, maintaining the last moved direction and always turning left if the next cell is not blue. We treat the sides of the board as colored stones for this purpose.

Using this traversal method, we start from the southwest corner and trace a path between cells that always keeps blue stones on the left. We continue until we reach the east side or the north side. No other outcome is possible. If we reach the east side, there are blue stones connecting the west and east sides. Otherwise, there are not.

If there is a connection from west to east, we now trace a path starting from the northwest corner, this time keeping blue stones to the right. The result will be two paths that enclose a connected set of blue stones from west to east.

The following diagrams illustrate this approach. We start by padding the board with colored stones to simplify the path-tracing procedure. You can convince yourself that this does not affect the outcome.

Now we must decide whether Blue has won or the board state is `Impossible`. To do this, we look for a blue stone that Blue could have placed to win the game. If the two paths that we traced from west to east share a stone, we can break the west-to-east connection by removing that stone. This means that Blue could have placed that stone and won the game, so the state is `Blue wins`. If the two paths do not share a stone, there is no single stone that Blue could have placed to win, so the state is `Impossible`.

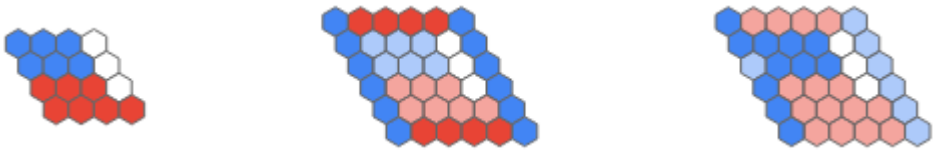
If there is no blue path, we repeat the above procedure for red paths from north to south. If there are no red paths either, the game state is `Nobody wins`.

In the example above, notice that the two paths share a stone in the top right cell of the original board. By removing this stone, it is clear that both paths are broken, so this must have been the

last stone that Blue placed. The state is `Blue wins`.

In the example below, notice that the paths do not share any stones. There is no stone that we can remove to break the connection. As discussed above, this state is `Impossible`.

In the final example, there is no path for Blue or Red. The state is `Nobody wins`:



It takes $O(\mathbf{N}^2)$ time to trace one path and we trace fewer than four paths, so the total time complexity of the path-tracing approach is $O(\mathbf{N}^2)$.

Kick Start 2022 - Coding Practice with Kick Start Session #1

Analysis: Milk Tea

View problem and solution walkthrough video

We are given two sets of binary strings:

1. Set of preferences of size $\mathbf{N}$
2. Set of forbidden milk teas of size $\mathbf{M}$

Each binary string in these sets is of length $\mathbf{P}$.

Test Set 1

There are only $2^{\mathbf{P}}$ different combinations of milk tea available. Since $\mathbf{P}$ is at most 10 for this set, the maximum number of possible combinations for a milk tea is 2^{10} or 1024. This is a small enough number to check each combination and calculate how many complaints you will get. Take the combination that gives the fewest complaints and that is not forbidden and output the number of complaints.

Test Set 2

For the this test set, the above approach will take too long to compute, so we need a different strategy.

Notice the following:

- Each option is represented by a bit. We select the best string of bits that fits the requirements.
- Disregarding forbidden combinations, a decision for which bit to choose for an option is independent of other bit decisions. This is because the options are independent from each other - every option will have a complaint or not for each friend, regardless of other options picked.
- Disregarding forbidden combinations again, we can get the best milk tea combination by selecting the most occurring bit in the set of preferences for each position in the string of bits. For example, given the set of preferences:
  ```
  { 1100
    1010
    0000 } the best milk tea would be:
    1000
  ```

So how do we deal with forbidden combinations? First notice that there are at most 100 forbidden combinations. The size of the forbidden set is $\mathbf{M}$; if we make $\mathbf{M}+1$ milk tea combinations, at least one of these combinations is not in the set of forbidden combinations. Since $\mathbf{M}$ is at most 100, it is feasible to create the best $\mathbf{M}+1$ combinations and take the best one that is not forbidden.

To generate these combinations, we can preprocess how many complaints we would get for each option. Then, we build up the best combinations one option at a time. We iterate through the options, at each option adding both possibilities (0 and 1) to the previous set of binary strings we created and select the best $\mathbf{M}+1$ results. We can tiebreak the results arbitrarily.

Let's consider why this works. Let S_k denote the best $\mathbf{M}+1$ combinations when we consider only the first k options. The key idea is to note that each combination in S_{k+1} has a combination from S_k as a prefix.

This is easy to show by contradiction. Take any binary string X from S_{k+1} and remove the last bit to get the prefix X'. Suppose, for contradiction, that X' is not S_k, thus all strings in S_k must have fewer complaints than X'. Take any of these strings in S_k and append the removed bit. This gives a combination which will have strictly fewer (when considering the tiebreak) complaints than X. This results in $\mathbf{M}+1$ binary strings with fewer complaints than X, which contradicts S being in S_{k+1}.

The final algorithm is as follows:

1. Precompute the scores (i.e. the number of complaints) for setting each bit in $\mathbf{P}$ to 0. The scores for setting a bit to 1 can then be computed by subtracting the score for 0 from $\mathbf{N}$, so we do not have to store these. To do this, iterate through 0 to $\mathbf{P}-1$, and at each iteration, iterate through the set of preferences, summing the number of 1s at that position.
2. Start with an empty set S_0: $S_0 = \{\text{""}\}$
3. Iterate from 1 to $\mathbf{P}$, generating S_k. To generate S_k (for $k > 0$):
    i. Take each binary string in S_{k-1} and append both a 0 and a 1. Since the set S_{k-1} has at most $\mathbf{M}+1$ strings, this will give at most $(\mathbf{M}+1)\cdot 2$ potential answers.
    ii. Remove extra strings so that the set has (at most) the best $\mathbf{M}+1$. To do this, you can maintain a dictionary of precomputed scores for each binary string in S_{k-1} and add the score for the appended bit to it. Then you can sort these scores and remove the largest ones.
4. Take the best combination from $S_{\mathbf{P}}$ that is not forbidden.

Time complexity

1. Precomputing the scores for each bit: $O(\mathbf{PN})$
2. Generating the empty set: $O(1)$
3. Iterating from 1 to $\mathbf{P}$: $O(\mathbf{P})$; in total for generating S_k: $O(\mathbf{M}\log\mathbf{M})$. So the total for generating all sets: $O(\mathbf{PM}\log\mathbf{M})$
    i. For each string in S_{k-1}: $O(\mathbf{M})$
        a. Appending 0 and 1: $O(1)$
        b. Computing the score for these two binary strings (remember, these are precomputed, you just need to add the score for the last bit to the score from the dictionary) and storing them in a dictionary takes: $O(1)$
    ii. Sorting the set and removing extra strings: $O(\mathbf{M}\log\mathbf{M})$
4. Finding the best combination from $S_{\mathbf{P}}$ that is not forbidden: $O(\mathbf{M})$

Thus, the total time complexity is $O(\mathbf{PN}+\mathbf{PM}\log\mathbf{M}+\mathbf{M})$.

Kick Start 2022 - Coding Practice with Kick Start Session #1

Analysis: Sample Problem

Solving this problem involves a number of steps. First, we must determine the total number of candies we have available to distribute. We can calculate this by summing together the number of candies in each bag, $total := \sum_{i=1}^{\mathbf{N}} \mathbf{C_i}$. Once we have the total number of candies, we must determine the number of candies that will remain after evenly distributing as many as possible among $\mathbf{M}$ kids. We can calculate this by dividing the total number of candies by the number of kids. $\mathbf{M}$ may not evenly divide $total$, so $(total \div \mathbf{M})$ will have an integer part (the amount each kid receives) and a remainder part (the amount left over). Because we only care about the remainder, we can use the modulo operation which does exactly this.

Example solution:

```
Solve(N, M, C) {
  total = 0
  for i in 1:N {
    total = total + C[i]
  }
  remainder = modulo(total, M)
  return remainder
}
```

Limits and complexity:

With the input limits for this problem, the default integer size for most programming languages will be sufficient to perform all arithmetic without risk of overflow. Specifically, the largest possible value is $total = (\mathbf{N_{max}} \times \mathbf{C_{imax}}) = 10^8$, which is easily stored in a 32-bit signed integer ($int_{max} \approx 2.147 \times 10^9$). If the limits were larger, we could take advantage of the transitive and symmetric properties of modular arithmetic to calculate the remainder provided by each bag, and the running remainder after each addition:

```
Solve(N, M, C) {
  remainder = 0
  for i in 1:N {
    remainder = remainder + modulo(C[i], M)
    remainder = modulo(remainder, M)
  }
  return remainder
}
```

This reduces the largest intermediate value to $2 \times (\mathbf{M_{max}} - 1)$ with only a small amount of extra computation. When solving other problems in this competition, you are likely to encounter limits that will require this kind of optimization.

Computational complexity is also an important factor to consider when deciding how to approach a problem. Both approaches presented here have the same complexity, $O(\mathbf{N})$, which is sufficient to solve the problem within the time limit. Other problems in this competition can often be solved using a variety of approaches, but only some will be sufficiently fast to solve all

test sets within the time limit. An algorithm with complexity $O(\mathbf{G}^3)$ may work for a small test set with $\mathbf{G}_{max} = 20$, but that same algorithm will be too slow for a large test set with $\mathbf{G}_{max} = 10^6$.

In general, time and memory limits for the problems in this competition, and the bounds for their test sets, are chosen so that algorithms pass or fail based on their overall time and memory complexity order. In other words, if a problem has a time limit of 60 seconds, it is unlikely that a fast algorithm would take 40 seconds while a slow one took 80 seconds. Instead, you will find that a fast algorithm might take 5 seconds while a slow one takes 5 years. So if you find your solution exceeding the time limit for a problem, take a moment to reevaluate your algorithm. There may be another approach that is many times faster.